

INSTITUTO DE INTELIGÊNCIA ARTIFICIAL APLICADA — I2A2

Protege Antes Ferramenta Inteligente para Comunicação Proativa com o Segurado

Relatório técnico · Desafio 5

Grupo: Cortex

Representante: Marcelo Barros de Azevedo Vieira — marcelobazevedo@gmail.com

Integrantes:

Fabício Oliveira Magalhães de Barros — fabriciotrabalho01@gmail.com

Luiz Eduardo Santos Lopes — dgjheuh@gmail.com

Alan Heverton Lima Martins — alanheverton7@gmail.com

Versão: 1.0

Data: agosto de 2026

Licença: MIT

Natureza: protótipo educacional com segurados fictícios e previsão real

1. Resumo executivo

O Protege Antes é um MVP que transforma o relacionamento entre seguradora e segurado de reativo em preventivo. O sistema consulta previsões meteorológicas externas, identifica eventos potencialmente danosos, cruza o risco com a localização e as apólices dos segurados, gera orientações personalizadas com um modelo de linguagem e simula a notificação.

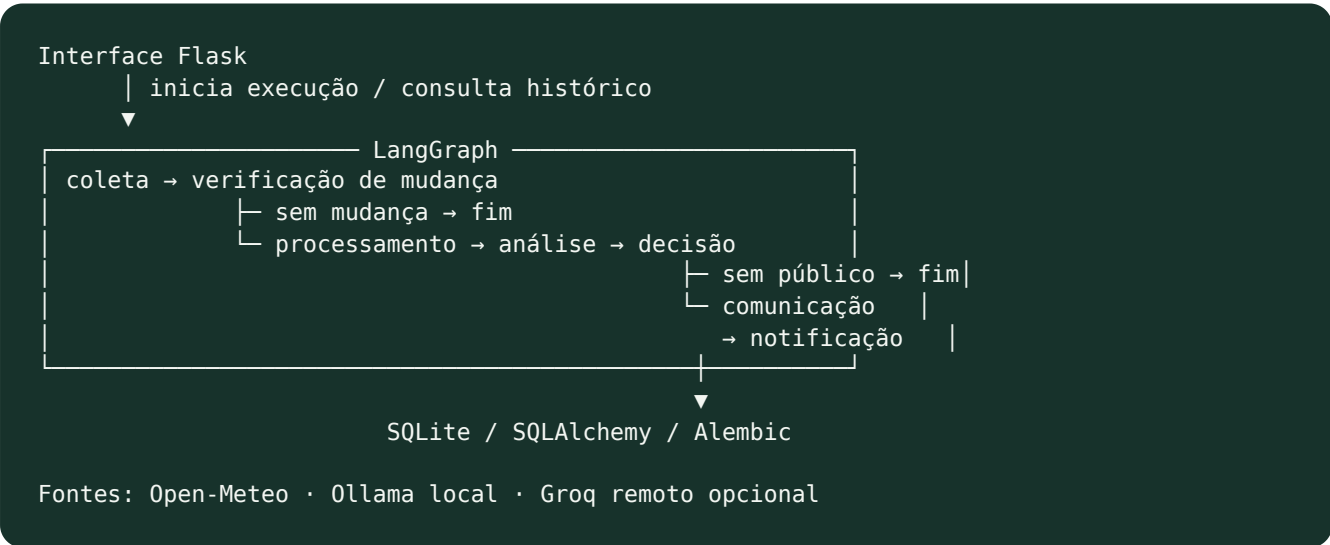
A solução demonstra um fluxo multiagente com LangGraph, modelo local Ministral 3 por Ollama, integração configurável com a Groq, persistência SQLite com SQLAlchemy e Alembic, interface Flask e ambiente Docker de desenvolvimento. Não há envio real nem uso de dados pessoais reais. A previsão é obtida da API pública Open-Meteo; segurados e exemplos de mensagens são fictícios.

2. Objetivos e escopo

O objetivo principal é antecipar riscos meteorológicos e comunicar ações preventivas antes de um sinistro. O MVP cobre chuva intensa, vento forte e granizo, seguros residencial e automóvel, previsão real pelo Open-Meteo e execução automática orientada a mudanças.

Fora do escopo: integração com sistemas reais de seguradoras, envio efetivo por WhatsApp/SMS/e-mail, interpretação contratual de cobertura, operação comercial e cobertura de todos os eventos climáticos.

3. Arquitetura da solução



A separação por camadas impede que interface, regras, integrações e persistência sejam misturadas. O Flask apresenta e inicia o caso de uso; o LangGraph coordena o estado; regras de domínio determinam o risco; adaptadores tratam APIs e banco.

3.1 Estado do grafo

O estado tipado contém identificador da execução, segurados, previsões, indicador de mudança, eventos, destinatários, mensagens e erros. Arestas condicionais encerram o fluxo quando a previsão não mudou ou quando não há público elegível, evitando processamento e chamadas desnecessárias à LLM.

4. Agentes especializados

Agente	Responsabilidade	Saída
Coleta	Consulta a previsão real no Open-Meteo.	Previsões associadas aos segurados.
Verificação	Compara o resumo SHA-256 normalizado com a última coleta.	Indicador auditável de mudança.
Processamento	Normaliza e elimina registros duplicados.	Previsões internas consistentes.
Análise de risco	Aplica limiares meteorológicos determinísticos.	Eventos, severidade e evidência.
Decisão	Cruza evento, localidade e tipo de apólice.	Decisões positivas e negativas justificadas.
Comunicação	Gera e valida JSON estruturado com Ollama/Groq.	Mensagem personalizada.
Notificação	Simula o envio e registra o resultado.	Notificação auditável no SQLite.

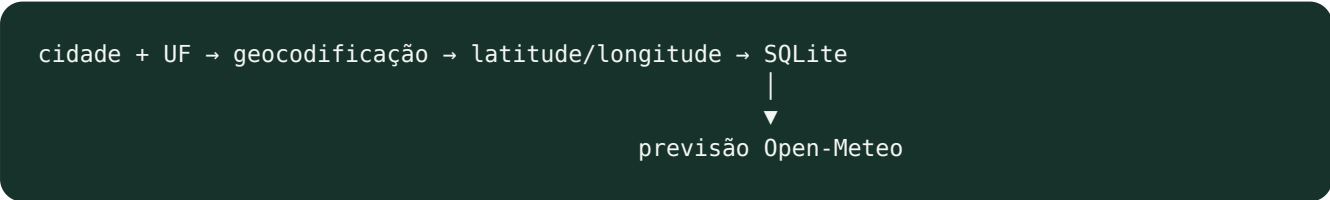
A LLM não decide se existe risco nem quem será notificado. Essas decisões permanecem explicáveis e testáveis nas regras de domínio.

5. Tecnologias utilizadas

Tecnologia	Uso
Python 3.13 e uv	Aplicação, dependências reproduzíveis e comandos.
LangGraph	Estado, nós, arestas e ramificação do fluxo.
Ollama + ministral-3:14b	Geração local de mensagens quando LOCAL=True.
Groq + openai/gpt-oss-20b	Alternativa remota configurável quando LOCAL=False; o uso real requer uma chave da Groq.
Open-Meteo	Previsão diária e geocodificação de cidade/UF.
Flask e Jinja	Painel, cadastro e auditoria visual.
SQLite, SQLAlchemy e Alembic	Persistência, ORM e evolução do esquema.
Docker Compose	Ambiente de desenvolvimento com volumes e código montado.
pytest, Ruff e Pyright	Testes, análise estática e verificação de tipos.

6. Fontes externas e geocodificação

O usuário cadastra nome, e-mail, telefone com DDD, cidade, UF, apólices e canal preferido. O telefone é normalizado com o código do Brasil para permitir uma futura integração com WhatsApp. A API de geocodificação do Open-Meteo procura a cidade no Brasil, valida a unidade federativa e retorna nome canônico, latitude e longitude. As coordenadas são persistidas e usadas pela API de previsão, que exige localização geográfica. Cada card permite editar nome, cidade, UF e apólices. Uma mudança de cidade ou UF refaz a geocodificação e atualiza as coordenadas. A exclusão lógica exige confirmação: o registro fica inativo e não participa de novas análises, enquanto notificações e execuções antigas permanecem auditáveis.



A previsão contempla até três dias. Depois da coleta, registros ordenados e sem duplicidade formam um resumo SHA-256. Se ele não mudou, o fluxo termina antes das regras, da LLM e das notificações.

7. Regras de negócio

Evento	Regra do MVP	Severidade	Apólices
Chuva intensa	≥ 50 mm, ou ≥ 30 mm com probabilidade $\geq 80\%$	Alta a partir de 50 mm	Residencial e automóvel
Vento forte	Rajada ≥ 75 km/h	Alta a partir de 90 km/h	Residencial e automóvel
Granizo	Código meteorológico WMO 96 ou 99	Alta	Automóvel

Os limiares são premissas transparentes do protótipo e não recomendações operacionais. Uma implantação real exigiria homologação por meteorologia, atuária e área de seguros.

Todos esses valores são carregados do `.env`: acumulados e probabilidade de chuva, limiares de vento e códigos WMO de granizo. A configuração é validada e convertida em um objeto imutável, injetado no agente de análise. Portanto, os parâmetros podem ser calibrados sem editar a regra de domínio ou reconstruir a imagem Docker.

7.1 Parâmetros configuráveis no ambiente

Variável do <code>.env</code>	Padrão	Descrição
ALERTA_CHUVA_INTENSA_MM	50 mm	Acumulado diário que dispara diretamente chuva intensa e severidade alta.
ALERTA_CHUVA_COM_PROBABILIDADE_MM	30 mm	Acumulado mínimo para alerta moderado quando combinado à probabilidade mínima.
ALERTA_PROBABILIDADE_MINIMA_CHUVA	80%	Probabilidade exigida para usar o limiar condicionado de precipitação.
ALERTA_VENTO_FORTE_KMH	75 km/h	Rajada mínima que caracteriza evento de vento forte.
ALERTA_VENTO_SEVERIDADE_ALTA_KMH	90 km/h	Rajada a partir da qual o vento forte recebe severidade alta.
ALERTA_CODIGOS_WMO_GRANIZO	[96,99]	Lista JSON de códigos meteorológicos WMO interpretados como trovada com granizo.

Os acumulados e as velocidades devem ser positivos; a probabilidade deve permanecer entre 0 e 100. O limiar condicionado de chuva não pode superar o de chuva intensa, e o limiar de severidade alta do vento não pode ser inferior ao de vento forte. Uma combinação inválida interrompe a inicialização com mensagem explícita, evitando decisões silenciosamente incoerentes.

```
# Exemplo no .env
ALERTA_CHUVA_INTENSA_MM=50
ALERTA_CHUVA_COM_PROBABILIDADE_MM=30
ALERTA_PROBABILIDADE_MINIMA_CHUVA=80
ALERTA_VENTO_FORTE_KMH=75
ALERTA_VENTO_SEVERIDADE_ALTA_KMH=90
ALERTA_CODIGOS_WMO_GRANIZO=[96,99]

# Aplicar alterações sem rebuild
docker compose up -d --force-recreate aplicacao
```

8. Geração das mensagens

A configuração LOCAL seleciona o provedor. O prompt solicita português brasileiro, texto curto, tom empático, recomendação prática, ausência de alarmismo e nenhuma promessa de cobertura. Ollama e Groq recebem JSON Schema, e Pydantic valida a resposta. As tentativas são configuráveis; se o provedor falhar, uma mensagem de contingência é produzida e a ocorrência fica registrada. A integração com a Groq está implementada e possui validação de resposta e testes automatizados, mas uma chamada externa real depende do fornecimento de uma chave válida.

As mensagens abaixo são exemplos ilustrativos produzidos para documentar o comportamento esperado. Os nomes são fictícios; em operação, as condições meteorológicas vêm do Open-Meteo.

Exemplo — granizo e seguro automóvel

Olá, Ana! Há previsão de granizo em São Paulo em 19/08/2026. Se possível, guarde o veículo em local coberto e evite permanecer próximo a árvores. Esta é uma orientação preventiva e não representa confirmação de cobertura. Em emergência, siga as autoridades locais.

Exemplo — chuva e seguro residencial

Olá, Ana! Há previsão de chuva intensa em São Paulo em 19/08/2026, com acumulado estimado de 62 mm. Proteja objetos em áreas baixas e não atravesse vias alagadas. Em emergência, acompanhe as orientações dos órgãos oficiais.

9. Persistência e auditoria

O SQLite contém as tabelas segurados, apolices, execucoes, eventos_climaticos, mensagens, notificacoes, verificacoes_meteorologicas, consultas_meteorologicas, decisoes e auditorias_agentes. As migrações 0001 a 0003 criam a base, a localização e a exclusão lógica; a 0004 registra as verificações; a 0005 acrescenta o canal da notificação; e a 0006 normaliza a origem; a 0007 acrescenta preferências e auditoria; a 0008 separa mensagens; e a 0009 adiciona os contatos do

segurado e o destino da notificação. O painel apresenta totais, histórico, eventos, evidências, destinatário, mensagem, canal, horário, provedor e status simulado. O botão **Excluir todas** remove em uma transação notificações, eventos, verificações e execuções, após confirmação, mas preserva segurados e apólices. A limpeza é bloqueada enquanto houver uma execução em andamento.

10. Interface e operação

1. Acessar o painel em `http://localhost:5001`.
2. Consultar, cadastrar ou editar um segurado por cidade e UF.
3. Usar **Verificar agora** para consultar a previsão real.
4. Inspeccionar a mudança, os riscos e as mensagens na auditoria da execução.
5. Consultar as execuções automáticas; ausência de mudança ou alertas é um resultado válido.

A ausência de mudança ou de eventos acima dos limiares é tratada como resultado válido: nesses casos, o sistema encerra o ramo correspondente sem chamar a LLM nem criar notificações desnecessárias.

11. Docker e execução

Os serviços Flask e monitor executam em contêineres; Ollama e o modelo já baixado permanecem no host. O código é montado em `/aplicacao`, permitindo recarga sem rebuild. Volumes distintos preservam ambiente virtual, cache uv e SQLite. A rede do host permite alcançar Ollama em `127.0.0.1:11434`.

```
ollama list
docker compose up --build      # somente na primeira vez
docker compose up              # usos seguintes
docker compose logs -f aplicacao
docker compose logs -f monitor
docker compose exec aplicacao uv run pytest -q
docker compose exec aplicacao uv run ruff check .
docker compose exec aplicacao uv run pyright
```

11.1 Monitoramento e simulação de notificação

O monitor consulta imediatamente ao iniciar e repete a verificação a cada 30 minutos por padrão. `MONITORAMENTO_ATIVO` liga ou desliga o serviço lógico, e `INTERVALO_MONITORAMENTO_MINUTOS` ajusta a frequência. A notificação é persistida, mas não enviada externamente. `CANAL_NOTIFICACAO` aceita whatsapp, sms ou email; a auditoria exhibe canal, destino, status e horário. O WhatsApp permanece simulado, mas o telefone normalizado prepara a futura integração com a Evolution API. O histórico marca como **Manual** as execuções iniciadas por botão ou CLI e como **Automática** as iniciadas pelo monitor. A migração 0006 normaliza os registros anteriores para essa classificação. O painel aberto consulta novamente seu estado a cada 15 segundos; uma execução manual exhibe um indicador de atividade e bloqueia cliques duplicados.

Falhas temporárias no Open-Meteo recebem até três tentativas. Uma falha parcial produz o status **concluída com ressalvas** e preserva o processamento das demais cidades; ausência total de previsões produz **erro**. O prompt pede texto simples, e a saída da LLM é normalizada para remover marcadores Markdown antes de persistir e exibir.

Todos os instantes são persistidos em UTC. Na apresentação, um filtro central interpreta como UTC os valores sem fuso devolvidos pelo SQLite e converte execuções e notificações para America/Sao_Paulo. A correção também alcança o histórico sem reescrever o banco.

Cada formulário POST possui token CSRF. Cada segurado escolhe WhatsApp, SMS ou e-mail. A auditoria exibe a linha do tempo dos agentes, as decisões negativas e positivas e cada tentativa de consulta meteorológica, incluindo fonte, parâmetros, duração, status e erro.

12. Verificações e resultados

A qualidade é verificada por testes automatizados, Ruff e Pyright. O funcionamento também pode ser acompanhado pela interface, pelos logs e pela trilha persistida no SQLite.

pytest	15 aprovados
Ruff	sem erros
Pyright	sem erros
Alembic	0009 (head)
Flask	HTTP 200
Ollama no host	ministral-3:14b acessível pelos contêineres
Groq remoto	integração implementada; chave externa não incluída

13. Segurança, ética e limitações

- Todos os segurados são fictícios, o `.env`` não é versionado e formulários usam CSRF.
- As mensagens são simuladas e não substituem Defesa Civil ou órgãos meteorológicos.
- O sistema não promete cobertura ou indenização.
- Previsões possuem incerteza e podem divergir das condições observadas.
- Em produção seriam necessários consentimento, opt-out, controles LGPD, autenticação, autorização, servidor WSGI, monitoramento e homologação das regras.

14. Conclusão

O Protege Antes demonstra o fluxo solicitado desde a obtenção de dados externos até a comunicação personalizada. A arquitetura multiagente deixa coleta, processamento, decisão e comunicação claramente separadas. O uso local do modelo reduz dependência externa, enquanto o Groq oferece uma alternativa remota mediante configuração de credencial. A interface e a auditoria tornam as decisões compreensíveis e adequadas a uma apresentação didática.

Projeto licenciado sob MIT. A entrega inclui o relatório técnico em PDF e o código-fonte com README, configuração de exemplo, migrações, testes e instruções de execução. O arquivo `.env`, bancos locais, caches e credenciais não integram o pacote.